A framework for hyperparameter configuration in the context of software defect density prediction using bio-inspired optimization algorithms

Juan Murillo-Morera[1], Rubén Fuentes-Fernández[2], Andrés Calvo Vargas[3], Alessandro Cambronero Delgado[3]
Janio Jadán-Guerrero[4], Susana Alexandra Arias[5]
[1]School of Informatics and Computing, National University of Costa Rica
[2]Institute of Knowledge Technology & Research Group on Agent-based, Social Interdisciplinary Applications, University Complutense of Madrid, Spain
[3]School of Informatics and Computing, National University of Costa Rica
[4]Research Center for Human Sciences and Education (CICHE), Indo-America Technological University, 170103, Quito, Ecuador
[5]International University of La Rioja, Spain
[1]juan.murillo.morera@una.cr, [2]ruben@fdi.ucm.es, [3]{andres.calvo.vargas, alessandro.cambronero.delgado}@est.una.ac.cr, [4]janiojadan@uti.edu.ec, [5]susanaalexandra.arias-externo@unir.net

2025 IEEE 7th International Conference on BioInspired Processing (BIP) | 979-8-3315-7014-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/BIP68491.2025.11489144

***Abstract*—Software quality assurance is a fundamental concern in software engineering, particularly in the context of defect prediction. Automated hyperparameter optimization has proven to be an effective approach for enhancing the performance of predictive models, enabling early defect detection while reducing development costs and delivery time. This study proposes a defect density prediction framework that integrates bio-inspired optimization algorithms, specifically genetic algorithm to fine-tune model parameters. An empirical evaluation was conducted involving 140 experimental runs derived from 70 unique configurations, combining 10 datasets, 7 machine learning algorithms, and a single attribute selector, each evaluated using two optimization methods: Genetic and Exhaustive search. The results, supported by statistical analysis, demonstrate that the proposed framework exhibits stable performance across configurations. Both optimization techniques achieved comparable results, with no statistically significant differences observed, indicating that the framework is robust regardless of the chosen optimization strategy.**

***Index Terms*—Defect prediction models, Learning schemes, Software metrics, Software quality, Statistical analysis, Empirical procedure**

I. Introduction

Software reliability is a critical concern in software engineering [1]. With the growing dependence on software systems across various domains, ensuring software quality has become increasingly important. As noted by Podgorelec [1], "The early and accurate identification of potentially dangerous (faulty) software modules is of vital importance for better software reliability." To enhance software quality and optimize the use of available resources, defect density prediction models typically rely on software metrics. Numerous studies have explored this research area [2], [3], [4], [5].

Research on software fault prediction can generally be categorized into three main approaches [6]: (1) estimating the number of defects remaining in a software system; (2) discovering associations among defects; and (3) classifying software components as defect-prone or not. According to Song et al. [6], the first approach often uses statistical techniques to estimate the number of defects or the defect density of source code. The second approach utilizes association rule mining to discover patterns among software defects. The third approach relies on classification models to identify components likely to contain defects. This work focuses on the first category, which remains a challenging and partially unsolved problem.

Several widely recognized datasets, including those from the NASA-MDP and PROMISE repositories, have been used in previous studies on defect prediction [6], [7], [8], [9]. These datasets often require preprocessing due to varying levels of noise. As emphasized by Shepperd et al. [10], applying preprocessing techniques is essential to handle missing values, outliers, and other inconsistencies.

In recent years, the defect prediction approach has increasingly adopted the use of *hyperparameters* [6]. At a high level, hyperparameter tuning involves a three-step process: (1) data preprocessing, (2) attribute selection, and (3) applying machine learning algorithms to optimize hyperparameter configurations. This research specifically addresses the challenge of selecting the best-performing hyperparameter combinations in order to maximize the performance of the predictive model for a given dataset. To achieve this, a genetic algorithm-based optimization strategy is used.

Following the Goal-Question-Metric (GQM) paradigm [11], the objective of this research is formally stated as follows:

Goal: To develop and evaluate a framework for software defect

979-8-3315-7014-9/25/$31.00 © 2025 IEEE

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.

density prediction using bio-inspired optimization algorithms. ***Apply:*** a bio-inspired optimization-based prediction framework ***For the purpose of:*** comparing performance across different hyperparameter configurations ***With respect to:*** datasets from both academic and industrial sources ***From the point of view of:*** researchers and software engineering practitioners ***In the context of:*** defect density prediction within the domain of software fault prediction.

The remainder of this paper is organized as follows: Section II presents the research questions; Section III discusses related and state-of-the-art approaches; Section IV describes the proposed framework; Section V outlines the experimental design; Section VI reports performance metrics, hypothesis testing, and statistical analysis; and finally, Section VII provides conclusions and directions for future work.

II. RESEARCH QUESTIONS

This study is guided by the following research questions, which aim to evaluate the performance, effectiveness, and computational efficiency of the proposed defect density prediction framework:

- **RQ1:** Which hyperparameter combinations yielded the best predictive performance (as measured by RMSE) under both exhaustive and genetic optimization approaches?
- **RQ2:** Which machine learning algorithms demonstrated the highest predictive accuracy based on RMSE?
- **RQ3:** Is there a statistically significant difference in execution time between the exhaustive and genetic optimization methods?
- **RQ4:** Does the genetic optimization approach achieve predictive performance (in terms of RMSE) comparable to that of the exhaustive search?

III. RELATED WORK

Numerous studies have advanced software defect prediction frameworks, often leveraging publicly available datasets such as NASA-MDP and PROMISE, which have become standard benchmarks for evaluating machine learning models.

In 2007, Menzies et al. [7] compared Rule Induction and Naive Bayes classifiers for identifying defect-prone components, while Zhang and Zhang [12] emphasized the relevance of precision and recall as key evaluation metrics.

In 2008, Lessmann et al. [13] conducted a large-scale empirical comparison of twenty-two classification algorithms across multiple software projects, providing one of the most comprehensive evaluations in the field. In 2010, Liu et al. [14] introduced a search-based approach for software quality modeling that enhanced model generalizability by integrating data from multiple repositories.

Building on these efforts, Song et al. [6] in 2011 examined twelve machine learning algorithms for fault prediction and found that Naive Bayes with logarithmic data transformation achieved the highest accuracy.

In 2015, Murillo et al. [15] proposed a genetic algorithm–based framework for defect prediction that outperformed earlier benchmarks. Their subsequent works in 2016 [8], [9] extended this approach to both effort estimation and defect prediction, further optimizing the use of evolutionary algorithms.

In 2018, Huda et al. [16] developed a hybrid framework combining Support Vector Machines (SVM) and Artificial Neural Networks (ANN) with maximum relevance filters for metric selection. This approach improved accuracy and reduced computational cost across eight PROMISE datasets.

That same year, Bashir et al. [17] addressed issues of high dimensionality, class imbalance, and noisy data through an integrated pipeline combining feature selection, data balancing, and noise filtering. Their experiments on twelve datasets (including SCM, OOM, and CombM) reported notable improvements in AUC scores.

Most recently, in 2024, Hesamolhokama et al. [18] introduced SDPERL, a reinforcement learning–based ensemble framework integrating CodeBERT and Proximal Policy Optimization (PPO) for automated feature selection. Experiments on PROMISE datasets showed a 6.25% increase in F1-score compared to conventional methods, highlighting the potential of combining deep learning and RL for defect prediction.

IV. PROPOSED FRAMEWORK

The proposed framework builds upon the work of Murillo et al. [8], [9] and comprises three main components: (1) *Generator*, (2) *Evaluator*, and (3) *Predictor*. It was implemented in Python using the Anaconda environment and the `scikit-learn` library [19].

As shown in Figure 1, the framework can be examined from two complementary perspectives: a *Black-Box* and a *White-Box* view.

The Black-Box view highlights the three core components that drive the iterative optimization process:

1) **Generator (Step 1):** Produces new hyperparameter combinations through genetic operators such as selection, crossover, and mutation.
2) **Evaluator (Step 2):** Measures the quality of generated solutions using fitness criteria and selects top-performing configurations through elitism.
3) **Predictor (Step 3):** Builds the predictive model with the best hyperparameter configuration to estimate defect density on new data.

The White-Box view details the internal workflow, which comprises three sequential processes. The first process, *Data Flow*, handles dataset preprocessing and partitioning. Raw data (DB) are transformed (DF, DF′) and divided into training (70%) and testing (30%) subsets. The second process, *Genetic Algorithm*, performs an evolutionary search to identify the optimal hyperparameter set. Chromosomes are iteratively refined until the best configuration is obtained. Finally, the *Prediction* process applies the selected configuration to unseen data, and the Predictor evaluates the model performance in the Evaluation stage.

This dual representation enhances both conceptual clarity and technical transparency. The Black-Box illustrates the modular design and interactions between components, while the White-Box exposes the operational details of data processing, optimization, and evaluation. Together, they provide a complete

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.

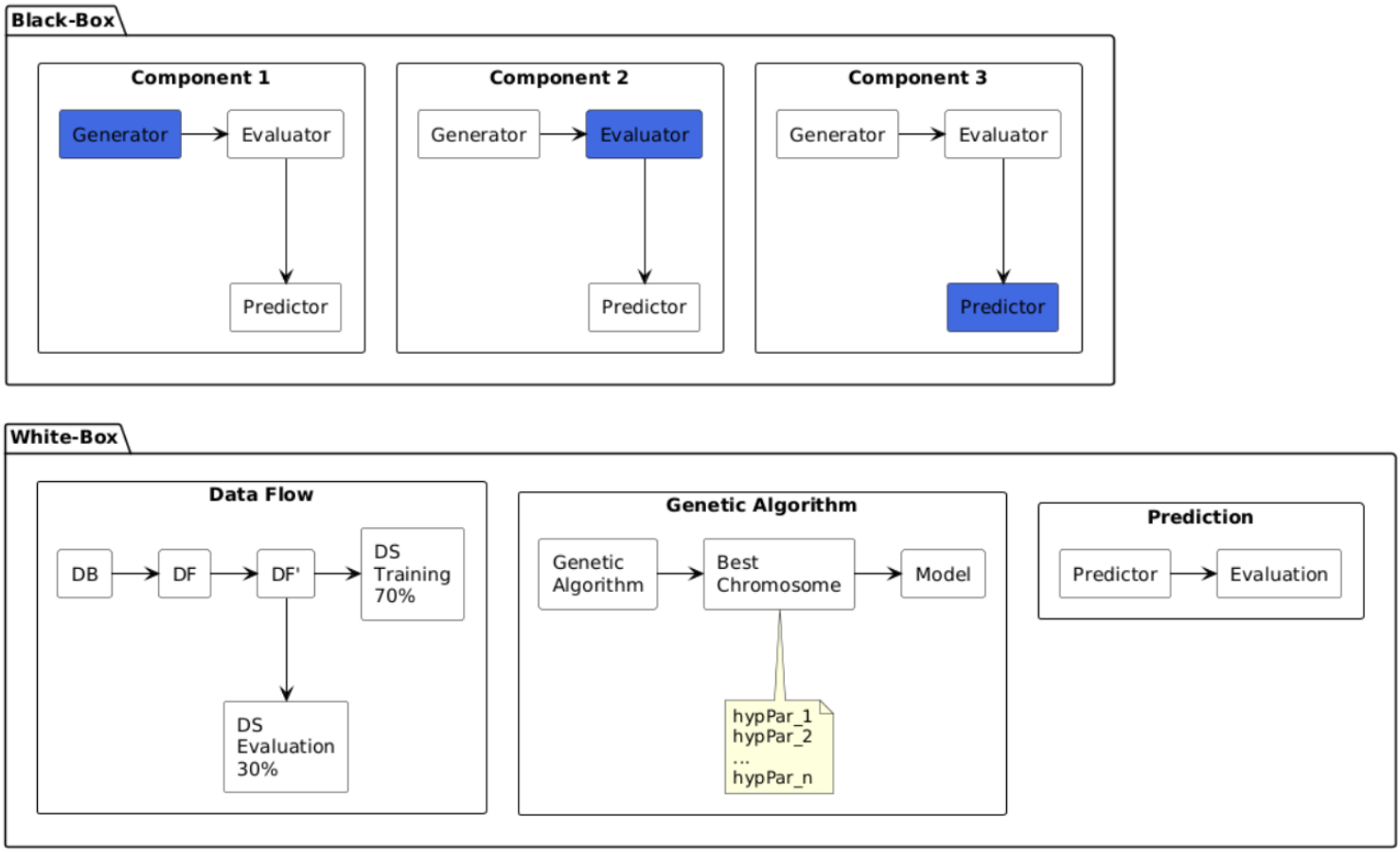


Fig. 1. Defect Density Prediction Framework based on [15], [8], [9]

understanding of the proposed defect density prediction framework.

The framework ultimately selects the hyperparameter configuration that achieves the best performance—identified as the optimal chromosome according to the fitness criteria.

The adoption of Genetic Algorithms (GAs) is motivated by their proven ability to efficiently explore large and complex search spaces through evolutionary principles [20]. Unlike exhaustive search, which becomes computationally infeasible with increasing dimensionality, GAs balance exploration and exploitation, promoting convergence toward high-quality solutions while reducing computational cost [8], [9].

Hyperparameter integration is essential because these parameters strongly influence model performance and generalizability. As noted by Bischl et al. [21], careful tuning of hyperparameters such as learning rate, tree depth, and regularization strength is crucial to avoid underfitting or overfitting. In defect density prediction—where datasets often contain noise and heterogeneity—systematic optimization enables robust and accurate models with manageable computational overhead.

Thus, combining GA-driven search with hyperparameter optimization ensures methodological rigor and empirical robustness, making the proposed framework a reliable tool for defect density prediction across diverse datasets.

A. Defect Prediction

The *Predictor* component (Figure 1) represents the final stage of the framework. As described by Murillo et al. [15], its objective is to build predictive models for new data using the selected chromosome and historical datasets.

This component applies trained models to unseen data and evaluates predictive accuracy. To ensure unbiased assessment, each dataset is split into 70% for training and 30% for testing, ensuring that testing data remain unseen during training.

Within the training subset, a 10-fold cross-validation procedure is applied. Both training and testing data undergo identical preprocessing and feature selection to maintain consistency.

After identifying the optimal hyperparameter configuration, the trained model is evaluated on the held-out data. Performance is measured using Root Mean Squared Error (RMSE) and the coefficient of determination (R^2).

The framework supports seven regression models commonly used in defect prediction: Linear Regression, Decision Tree Regressor, Random Forest Regressor, Lasso, Ridge, K-Nearest Neighbors, and XGBoost. To improve reliability and reduce variance, the evaluation is repeated ten times (N-PASS = 10) with different random splits. Final results are reported as the average across these runs.

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.

B. Genetic Operators

The framework employs standard GA operators—selection, reproduction, crossover, and mutation—following the default settings from the WEKA genetic search module [22]: number of generations = 20, population size = 20, crossover probability = 0.6, mutation probability = 0.01, and elitism for retaining top-performing individuals [8], [9].

V. Empirical Study

A. Datasets

To evaluate the proposed framework, ten datasets from the PROMISE Software Engineering Repository were used [23]. These include: *ant-1*, *poi-2*, *camel-1*, *tomcat*, *ivy-2*, *velocity-1*, *jedit-4*, *xalan-2*, *log4j-1*, and *xerces-1*. Each dataset contains object-oriented software metrics (e.g., CBO, DIT, and LCOM), and the target variable corresponds to the number of defects per software module. These datasets are structured for regression-based defect density prediction tasks. A comprehensive description of the datasets, including their features, instance counts, and project context, is provided in Table I (see Appendix). Further details can be found in [23].

B. Hyperparameter Configuration

As discussed in Section I, the hyperparameter space in our framework includes three key elements: (1) the model hyperparameters, (2) the attribute selection method, and (3) the learning algorithm. The configurations used for each component are described below.

- *Hyperparameters*:
  - LinearRegression: *copy_X*, *fit_intercept*, *positive*.
  - DecisionTreeRegressor: *max_depth*, *min-samples-split*, *min-samples-leaf*, *random_state*.
  - RandomForestRegressor: *n_estimators*, *max_depth*, *min_samples_split*, *random_state*.
  - Lasso: *alpha*, *fit_intercept*, *max_iter*, *tol*, *selection*.
  - Ridge: *alpha*, *fit_intercept*, *tol*, *solver*.
  - KNeighborsRegressor: *n_neighbors*, *weights*, *algorithm*.
  - XGBRegressor: *learning_rate*, *n_estimators*, *max_depth*, *subsample*, *colsample_bytree*.
- *Attribute selector (AS)*: This study employed a single attribute selection method: `SelectFromModel` with `LassoCV`, which performs feature selection based on the L1-regularization path.
- *Learning Algorithm (LA)*: LinearRegression, DecisionTreeRegressor, RandomForestRegressor, Lasso, Ridge, KNeighborsRegressor, XGBRegressor.

C. Experimental Design

The experimental protocol follows the methodology described by Murillo et al. [8], [9], adapted for regression-based defect prediction.

- **Search Space and Executions:** A total of 70 unique configurations were defined by combining 10 datasets, 7 learning algorithms, and 1 attribute selector (10 × 7 × 1). Each configuration was tested using two optimization strategies: Genetic and Exhaustive, resulting in 140 total executions.
- **Optimization Process:** Both optimization methods operated over the same configuration space. Genetic optimization used evolutionary strategies, while exhaustive optimization employed grid search.
- **Data Partitioning:** For each dataset, 70% of the data was randomly selected as training data (historical), while the remaining 30% was reserved for testing. This process was repeated 10 times (N-PASS = 10), with each pass involving different randomized partitions.
- **Preprocessing:** Missing values were removed, and the target variable (defects) was log-transformed using `log(defects + 1)` to reduce skewness. Standard scaling was applied to all features using `StandardScaler`.
- **Feature Selection:** The `SelectFromModel` method with `LassoCV` was applied uniformly across all datasets and runs. This approach was chosen for its computational efficiency and regularization-based interpretability.
- **Model Evaluation:** Each model configuration underwent a 10-fold cross-validation procedure. Performance metrics, including RMSE and R^2, were computed for each fold. Additionally, the model was evaluated on the held-out 30% test set. This dual evaluation strategy enabled the analysis of both generalization and predictive performance.
- **Robustness and Repetition:** The entire process (including training, tuning, and testing) was repeated 10 times per configuration, resulting in a total of 140 executions × 10 repetitions = 1,400 individual experiments. Final performance scores were averaged to reduce variance and improve statistical reliability.

This experimental design ensures a rigorous and reproducible evaluation of the framework across diverse datasets and learning strategies, enabling robust comparisons between the optimization methods.

VI. Results and Analysis

This section addresses the research questions outlined in Section II, based on the evaluation conducted using the ten datasets described in Section V.

- **RQ-1: What were the best hyperparameter combinations according to RMSE results for both the exhaustive and genetic approaches?**
  The optimal hyperparameter configurations varied slightly depending on the dataset and the learning algorithm. However, both optimization techniques identified configurations with highly similar RMSE values. For instance, in the `poi-2` dataset using the Lasso regressor, both genetic and exhaustive searches achieved an RMSE of 0.222. Nonetheless, the genetic method required 0.052 seconds to converge, whereas the exhaustive approach completed in just 2.486 seconds.
  These results indicate that the genetic search was capable of efficiently exploring the hyperparameter space and converging toward near-optimal configurations. While it matched the performance of the exhaustive search in

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.

many cases, it also provided flexibility to explore more complex or continuous search spaces. Complete results for all datasets and algorithms are presented in Tables II and III, with detailed comparisons summarized in Table II (see Appendix).

- **RQ-2: Which algorithms showed the best performance according to RMSE?**
  Based on the results in Table III and Table V, ensemble models such as `XGBRegressor` and `RandomForestRegressor` consistently outperformed other models across most datasets. These algorithms demonstrated lower RMSE values and better generalization in predicting defect density, especially in datasets with complex patterns and higher noise levels. Linear models such as `Lasso` and `LinearRegression` also performed competitively in specific datasets like `poi-2` and `xalan-2`, which exhibited lower variance and stronger linear correlations. These findings suggest that while ensemble methods generally provide superior performance, simpler linear models remain effective for certain data characteristics.
- **RQ-3: Is there any statistically significant difference in runtime between the exhaustive and genetic approaches?**
  A clear and statistically significant difference in execution time was observed. As shown in Table IV, the exhaustive search algorithm consistently required more time to complete compared to the genetic approach. For example, in the `ant-1` dataset with `LinearRegression`, the exhaustive search took 6.52 seconds, while the genetic method completed in just 0.05 seconds, representing a 130.4-fold reduction in time.
  To validate these observations, a Wilcoxon Signed-Rank Test was applied to compare runtimes. The null hypothesis (H_0) stated that there is no significant difference in execution time between the two methods; the alternative hypothesis (H_1) posited a significant difference. The resulting p-value was 0.0156, which is below the 0.05 threshold. Therefore, H_0 is rejected, confirming a statistically significant difference in runtime. The genetic approach is conclusively faster.
- **RQ-4: Did the genetic approach achieve similar RMSE results to the exhaustive approach?**
  To assess the equivalence of predictive performance, we again applied the Wilcoxon Signed-Rank Test, this time on RMSE results. The null hypothesis (H_0) was that there is no significant difference in RMSE between the two methods; the alternative (H_1) was that a difference exists. The resulting p-value was 0.1025, which exceeds the 0.05 threshold. Therefore, we fail to reject H_0.
  This outcome indicates that there is no statistically significant difference in RMSE between the genetic and exhaustive optimization strategies. Both methods achieve comparable levels of predictive performance across all evaluated datasets and models.

Construct Validity

This study employs RMSE and R^2 as evaluation metrics, commonly accepted for regression-based defect prediction. The use of attribute selection (`SelectFromModel` with `LassoCV`) and hyperparameter tuning follows established practices. While filter methods are computationally efficient, wrapper or hybrid strategies may improve accuracy and are considered for future work.

Conclusion Validity

Wilcoxon signed-rank tests were applied to assess differences between the optimization methods. The results (p = 0.0156 for runtime, p = 0.1025 for RMSE) support our conclusions, indicating a significant runtime difference but no significant difference in predictive performance. Future analyses may include additional statistical tests to enhance reliability.

VII. Conclusions and Future Work

This study presented an empirical evaluation of a software defect density prediction framework based on bio-inspired optimization techniques. Two hyperparameter optimization strategies, Genetic and Exhaustive search, were assessed across 70 unique configurations derived from the combination of 10 datasets, 7 machine learning algorithms, and a single attribute selection method. Each configuration was evaluated under both optimization strategies, resulting in a total of 140 executions.

The experiments employed `SelectFromModel` with `LassoCV` for attribute selection, and each configuration was evaluated using 10-fold cross-validation. Performance was assessed primarily through RMSE and R^2 metrics.

The results demonstrated that both optimization techniques yielded comparable predictive performance, with no statistically significant differences in RMSE. However, the Genetic approach significantly reduced computation time compared to the Exhaustive method. These findings confirm the stability, robustness, and efficiency of the proposed framework for software defect density prediction.

From a practical perspective, the framework offers a reliable and computationally efficient method for hyperparameter tuning in defect prediction tasks. It supports software engineers and researchers in identifying effective configurations to improve testing strategies, reduce development costs, and optimize project timelines.

Future work will focus on extending the evaluation to more diverse datasets, exploring additional machine learning models and attribute selection techniques, and refining the genetic algorithm configurations to further enhance robustness and scalability.

VIII. Acknowledgments

This research was supported by National University of Costa Rica (No. 0127-23).

IX. Appendix

Full tables are provided in the Supplementary Material [24].

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] V. Podgorelec, "Improved mining of software complexity data on evolutionary filtered training sets," *WSEAS Trans. Info. Sci. and App*, vol. 6, pp. 1751–1760, 2009.

[2] R. Malhotra, "Comparative analysis of statistical and machine learning methods for predicting faulty modules," *Applied Soft Computing*, vol. 21, pp. 286–297, 2014.

[3] P. Singh and S. Verma, "Empirical investigation of fault prediction capability of object oriented metrics of open source software," in *Computer Science and Software Engineering (JCSSE), 2012 International Joint Conference on*. IEEE, 2012, pp. 323–327.

[4] H. Wang, T. M. Khoshgoftaar, and A. Napolitano, "Software measurement data reduction using ensemble techniques," *Neurocomputing*, vol. 92, pp. 124–132, 2012.

[5] J. Murillo, C. Quesada, and M. Jenkins, "Software fault prediction: A systematic mapping study," in *XVIII Ibero-American Conference on Software Engineering*. CibSE, 2015, pp. 50–64.

[6] Q. Song, Z. Jia, M. Shepperd, S. Ying, and J. Liu, "A general software defect-proneness prediction framework," *Software Engineering, IEEE Transactions on*, vol. 37, no. 3, pp. 356–370, 2011.

[7] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," *Software Engineering, IEEE Transactions on*, vol. 33, no. 1, pp. 2–13, Jan 2007.

[8] J. Murillo-Morera, C. Quesada-López, C. Castro-Herrera, and M. Jenkins, "An empirical validation of an automated genetic software effort prediction framework using the isbsg dataset."

[9] J. Murillo-Morera, C. Castro-Herrera, J. Arroyo, and R. Fuentes-Fernández, "An Empirical Validation of Learning Schemes using an Automated Genetic Defect Prediction Framework," July 2016, in press Springer.

[10] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the nasa software defect datasets," *Software Engineering, IEEE Transactions on*, vol. 39, no. 9, pp. 1208–1215, 2013.

[11] V. R. Basili, "Software modeling and measurement: the goal/question/metric paradigm," 1992.

[12] H. Zhang and X. Zhang, "Comments on "data mining static code attributes to learn defect predictors"," *IEEE Transactions on Software Engineering*, vol. 33, no. 9, pp. 635–637, Sept 2007.

[13] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," *IEEE Transactions on Software Engineering*, vol. 34, no. 4, pp. 485–496, 2008.

[14] Y. Liu, T. M. Khoshgoftaar, and N. Seliya, "Evolutionary optimization of software quality modeling with multiple repositories," *IEEE Transactions on Software Engineering*, vol. 36, no. 6, pp. 852–864, 2010.

[15] J. Murillo-Morera and M. Jenkins, "A software defect-proneness prediction framework: A new approach using genetic algorithms to generate learning schemes," in *The 27th International Conference on Software Engineering and Knowledge Engineering, SEKE 2015, Wyndham Pittsburgh University Center, Pittsburgh, PA, USA, July 6-8, 2015*, 2015, pp. 445–450.

[16] S. Huda, S. Alyahya, M. M. Ali, S. Ahmad, J. Abawajy, H. Al-Dossari, and J. Yearwood, "A framework for software defect prediction and metric selection," 1 2018.

[17] K. Bashir, T. Li, and W. Chubato, "An empirical study for enhanced software defect prediction using a learning-based framework," *International Journal of Computational Intelligence Systems*, vol. 12, p. 282, 11 2018.

[18] M. Hesamolhokama, A. Shafiee, M. Ahmaditeshnizi, M. Fazli, and J. Habibi, "Sdperl: A framework for software defect prediction using ensemble feature extraction and reinforcement learning," 2024.

[19] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The weka data mining software: an update," *ACM SIGKDD explorations newsletter*, vol. 11, no. 1, pp. 10–18, 2009.

[20] Z. Jakšić, S. Devi, O. Jakšić, and K. Guha, "A comprehensive review of bio-inspired optimization algorithms including applications in microelectronics and nanophotonics," *Biomimetics*, vol. 8, no. 3, p. 278, Jun. 2023.

[21] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, L. Ullmann, J. Becker, M. Boulesteix, D. Deng, and M. Lindauer, "Hyperparameter optimization: Foundations, algorithms, best practices and open challenges," *arXiv preprint arXiv:2107.05847*, nov 2021.

[22] I. H. Witten and E. Frank, *Data Mining: Practical machine learning tools and techniques*. Morgan Kaufmann, 2005.

[23] T. Lamba, Kavita, and A. Mishra, "Optimal metrics selection for software defect prediction," *International Journal of Data Mining and Emerging Technologies*, vol. 7, p. 82, 1 2017.

[24] A. Calvo, "Appendix: Extended tables for the optimml framework," Available in the GitHub repository "OptimML-Framework", 2025, access via GitHub: AndresACV/OptimML-Framework.

Authorized licensed use limited to: Universidad Nacional de Costa Rica (UNA). Downloaded on May 25,2026 at 16:35:23 UTC from IEEE Xplore. Restrictions apply.